DAY 6 DSA

Complete Lecture-Based Study Notes

Topics: Hash Tables & Hashing • Collision Handling • HashMap Implementation • Trees • Binary Search Trees • Tree Traversals

How to use these notes

These notes follow the uploaded Day 6 lecture transcript. The lecture moves from hashing and collision handling into the introduction of trees, Binary Search Trees (BSTs), insertion logic, and recursive traversals. Small examples and code snippets are added to make the lecture easier to revise; where a snippet is a teaching example rather than a verbatim transcript, it is presented as an illustrative implementation.

Study order: understand the idea → trace the example by hand → explain the interview definition → practise the questions at the end.

1. Quick revision: stacks and queues

The lecture begins by revising the previous session: stack operations, infix/prefix/postfix conversion and evaluation, parenthesis balancing, linear/circular queues, deque, and priority queues. The new focus is hashing, followed by trees.

• **Stack:** LIFO — last in, first out.

• **Queue:** FIFO — first in, first out.

• **Deque:** insertion and deletion are allowed at both ends.

• **Priority queue:** elements are removed according to priority; heaps are a common efficient implementation, which the lecture connects to trees.

2. Hash table: core idea

Definition: A hash table is a data structure that stores information as key–value pairs and uses a hash function to map a key to a table slot so the corresponding value can be accessed quickly.

Example: a phone contacts app maps a person's name (key) to a phone number (value). A student-record example maps roll number / PRN (key) to a student object (value).

Important operations

• **put(key, value)** — insert or update the key–value pair.

• **get(key)** — retrieve the value associated with a key.

Other useful operations in real implementations include remove, containsKey, size, and keySet, but the lecture emphasizes put and get as the basic operations.

Why use hashing?

Technique	Typical search time	Main idea
Linear search	O(n)	Inspect elements one by one
Binary search	O(log n)	Repeatedly halve a sorted search range
Hash table	Average O(1)	Compute a slot from the key

Important qualification: O(1) is the ideal/average-case target, not a guarantee for every operation. Collisions and a poor hash function can make operations slower.

3. Table, slot, key, value, hash function

• **Table:** the storage structure (often backed by an array).

• **Slot / bucket:** a position in the table.

• **Key:** the identifier used to find data.

• **Value:** the data associated with that key.

• **Hash function:** a function that maps a key to a valid table index.

For roll numbers 1–50, a direct mapping can use index = rollNumber – 1. Roll number 5 goes to index 4, so both insertion and lookup can access that position directly.

```
index = key - 1 // direct mapping for keys 1..50
```

This is simple when keys fit a small, dense range. It becomes wasteful or impossible when keys are large and scattered, such as thousands of PRNs stored in a table with only 1,000 slots.

4. Hash functions and modulo

A common simple hash function uses the remainder operator (%):

```
index = key % tableSize
```

If tableSize = 1000, then 3002 % 1000 = 2, and 4998 % 1000 = 998. The result is always between 0 and tableSize – 1 for a positive table size.

Worked example: table size = 10. Keys 12, 22 and 32 all map to index 2 because each leaves remainder 2 when divided by 10. This leads to a collision.

5. Collision

Collision: two different keys produce the same table index under the hash function.

A perfect collision-free mapping is generally unrealistic when many possible keys must fit into a smaller table. Hash tables therefore need a collision-resolution strategy.

Two collision-resolution strategies in the lecture

Strategy	How it works	Where the data goes
Open addressing	When a slot is occupied, probe other slots using a rehashing/probing rule	Inside the table array
Separate chaining	Each table slot points to a collection (commonly a linked list) of entries with that hash index	Entries can be stored in a linked list

6. Open addressing and probing

In open addressing, if the calculated slot is occupied by another key, the algorithm searches for another available slot. The probing rule determines which slot to try next.

Linear probing

```
initialIndex = key % tableSize
if occupied:
try (initialIndex + 1) % tableSize
if still occupied:
try (initialIndex + 2) % tableSize
continue until found or an available slot is found
```

The modulo at the end wraps around: if the last index is 999 in a table of size 1000, the next index becomes (999 + 1) % 1000 = 0.

Other probing styles

• **Linear probing:** try consecutive slots; simple, but clusters can form.

• **Quadratic probing:** use quadratic offsets, such as h(k) + 1², h(k) + 2², and so on (with table-size handling).

• **Double hashing:** use a second hash function to determine the step size.

The lecture describes the probing rule as a rehashing function and the repeated search for a slot as the probing procedure.

Searching with open addressing

```
1. Calculate the initial slot using the hash function.
2. Check whether the key in that slot matches the requested key.
3. If it matches, return the value.
4. If it does not match, follow the probing rule.
5. Repeat until the key is found or the search can conclude that it is absent.
```

A real implementation must handle table wrap-around and termination carefully; it must not probe forever if the table is full.

7. Load factor

Load factor (α) measures how full the hash table is:

```
load factor = number of stored key–value pairs / number of table slots
```

Entries	Slots	Load factor
900	1000	0.9
1000	1000	1.0
1100	1000	1.1

With open addressing, the table needs an available slot for a new distinct key, so it cannot keep inserting new entries once all slots are occupied. A load factor below 1 is therefore required for a usable open-addressed table. Implementations often resize and rehash before the table becomes too full.

For separate chaining, a load factor greater than 1 is possible because a bucket can hold multiple entries in its linked list.

8. Separate chaining

In separate chaining, the table is an array of buckets. Each bucket may hold a linked list (or another collection) of entries whose keys map to that bucket.

```
bucketIndex = key % tableSize
bucket = table[bucketIndex]

// insert: search the bucket for the key
// if key exists: update its value
// otherwise: add a new key-value entry to the bucket
```

Example with tableSize = 10: keys 12, 22 and 32 all map to bucket 2. Instead of searching for a free array slot, all three entries can be stored in bucket 2's chain.

Searching with chaining

• Compute the bucket index using the hash function.

• Go to that bucket's linked list.

• Search the list for the exact key; this part is linear in the length of that bucket's chain.

Average performance can be close to O(1) when the hash function distributes keys well and the load factor is controlled. Worst-case lookup can become O(n) if many keys land in the same bucket.

9. Implementing a simple hash table in Java

The lecture walks through implementing a hash table and a key–value pair, including buckets backed by linked lists. The following is an illustrative simplified version for integer keys and string values; it is meant to show the central idea, not to reproduce every line of the instructor's code.

```
import java.util.LinkedList;

class Entry {
int key;
String value;

Entry(int key, String value) {
this.key = key;
```

```
this.value = value;
}
}

class SimpleHashTable {
private LinkedList<Entry>[] table;

@SuppressWarnings("unchecked")
public SimpleHashTable(int size) {
table = (LinkedList<Entry>[]) new LinkedList[size];
for (int i = 0; i < size; i++) {
table[i] = new LinkedList<>();
}
}

private int hash(int key) {
return Math.floorMod(key, table.length);
}

public void put(int key, String value) {
LinkedList<Entry> bucket = table[hash(key)];
for (Entry e : bucket) {
if (e.key == key) {
e.value = value; // update existing key
return;
}
}
bucket.add(new Entry(key, value));
}

public String get(int key) {
LinkedList<Entry> bucket = table[hash(key)];
for (Entry e : bucket) {
if (e.key == key) return e.value;
}
return null; // key not found
}
}
```

Trace the code

```
SimpleHashTable map = new SimpleHashTable(10);
map.put(12, "Asha"); // bucket 2
map.put(22, "Ravi"); // bucket 2: collision handled by chaining
map.put(12, "Asha S."); // same key: update value
System.out.println(map.get(12)); // Asha S.
System.out.println(map.get(22)); // Ravi
System.out.println(map.get(32)); // null
```

This simplified example uses chaining. Java's built-in HashMap has additional details such as resizing, hash spreading, equality handling, and implementation optimizations.

10. Hash table complexity summary

Operation	Average case	Worst case (simple model)
put	O(1)	O(n)
get	O(1)	O(n)
remove	O(1)	O(n)

For chaining, worst-case O(n) can occur if all entries end up in one bucket. For open addressing, long probe sequences can also make operations linear in the number of entries / slots.

11. Introduction to trees

Tree: a finite collection of nodes with one designated root node; the remaining nodes form disjoint subtrees of that root. A tree represents hierarchical relationships.

• Examples: folders inside folders, an organization chart, family relationships, and the hierarchy of a DOM.

• **Node:** stores data and references to child nodes.

• **Root:** the top node; it has no parent.

• **Edge / link:** connection between a parent and child.

• **Parent:** a node directly above another node.

• **Child:** a node directly below another node.

• **Siblings:** nodes that share the same parent.

• **Leaf node:** a terminal node with no children.

• **Subtree:** a node together with its descendants.

• **Empty/null tree:** a tree with no nodes.

• **Degree of a node:** number of children (subtrees) of that node.

• **Degree of a tree:** the maximum degree of any node in the tree.

12. Levels, depth, height

The lecture introduces node levels and depth, then explains height by counting edges upward from a leaf.

• **Depth of a node:** number of edges from the root to that node. Root depth is 0.

• **Height of a node:** number of edges on the longest downward path from that node to a leaf.

• **Height of a tree:** height of its root.

The lecture uses the convention that a single-node tree has height 0 and an empty tree has height −1. Be consistent with the convention your course or interviewer specifies; some materials define height differently.

13. Types of trees covered

Multiway (n-ary) tree

A node may have multiple children. A node may therefore need a list of child references.

Binary tree

A binary tree is a tree in which each node has at most two children, conventionally called left and right. A node structure typically stores data, a left reference, and a right reference.

```
class Node {
int data;
Node left;
Node right;

Node(int data) {
this.data = data;
this.left = null;
this.right = null;
}
}
```

Binary Search Tree (BST)

A BST is a binary tree organized by a comparison rule. In the common version used here, smaller values go to the left and larger values go to the right. Duplicate-key handling depends on the chosen implementation.

• All values in a node's left subtree are smaller than the node's value.

• All values in a node's right subtree are larger than the node's value.

• Each subtree must also satisfy the BST property.

14. BST insertion logic

The lecture traces insertion by creating a node, assigning it as root if the tree is empty, or traversing from the root to find the correct empty child reference.

```
insert(value):
newNode = Node(value)
if root is null:
root = newNode
return

current = root
while true:
if value < current.data:
if current.left is null:
current.left = newNode
return
current = current.left
else if value > current.data:
if current.right is null:
current.right = newNode
return
current = current.right
else:
// duplicate policy: ignore or update, as designed
return
```

Why use a traversal pointer such as current/trav? The root must remain the entry point to the whole tree. Moving the temporary pointer lets us inspect deeper nodes without losing the root reference.

Insertion dry run

Insert these values in order: 50, 30, 10, 90, 100, 40, 70, 60, 20.

```
50
/ \
30 90
/ \ / \
10 40 70 100
\ /
20 60
```

Each value is compared with the current node and the algorithm moves left or right until it finds a null child. The final shape depends on insertion order.

BST search and complexity

• At each node, compare the target with the node's value.

• If equal, the key is found; if smaller, move left; if larger, move right.

Search/insert time is O(h), where h is the tree height. A reasonably balanced BST can provide O(log n) operations; a badly skewed BST can degrade to O(n).

15. Tree traversals

Traversal means visiting every node in a defined order. The lecture focuses on inorder, preorder, and postorder and implements them recursively.

Traversal	Order	Mnemonic
Preorder	Node → Left → Right	Root first
Inorder	Left → Node → Right	Root in the middle
Postorder	Left → Right → Node	Root last

```
void preorder(Node node) {
if (node == null) return;
System.out.print(node.data + " ");
preorder(node.left);
preorder(node.right);
}

void inorder(Node node) {
if (node == null) return;
inorder(node.left);
System.out.print(node.data + " ");
inorder(node.right);
}

void postorder(Node node) {
if (node == null) return;
postorder(node.left);
postorder(node.right);
System.out.print(node.data + " ");
}
```

The base case is essential: when node == null, return. It stops recursion when a missing child is reached.

Traversal dry run

For this small tree:

```
2
/ \
1 3
```

• **Preorder:** 2 1 3

• **Inorder:** 1 2 3

• **Postorder:** 1 3 2

For a BST with distinct values, inorder traversal outputs values in ascending order.

16. Interview-ready definitions

What is hashing?
A technique that maps a key to a table index using a hash function, enabling fast key-based access.

What is a collision?
When two different keys map to the same hash-table index.

What is linear probing?
An open-addressing collision strategy that checks successive table slots until it finds a suitable slot.

What is separate chaining?
A collision strategy in which each table bucket stores a collection of entries sharing that bucket index.

What is load factor?
The ratio of stored entries to table slots.

Why is hashing average O(1)?
The hash function usually computes the target bucket directly, so only a small amount of work is expected when keys are distributed well.

What is a tree?
A hierarchical structure of nodes connected by parent–child relationships, with a designated root.

What is a BST?
A binary tree that maintains an ordering rule between each node and its left/right subtrees.

Why does inorder traversal of a BST sort values?
Because it visits the left subtree, then the node, then the right subtree, matching the BST ordering rule.

What is the base case in recursive traversal?
If the current node is null, return without visiting anything.

17. Practice questions

• 1. A table has size 10 and uses h(k) = k % 10. Find the bucket for keys 17, 27 and 37.

• 2. Define collision and name the two collision-resolution approaches covered.

• 3. A table stores 750 entries in 1,000 slots. Calculate its load factor.

• 4. Why can open addressing not insert a new distinct key when every slot is occupied?

• 5. In separate chaining, where do you search after calculating the bucket index?

• 6. State the BST insertion rule for values smaller and larger than the current node.

• 7. What are the preorder, inorder and postorder outputs for the tree with root 8, left child 3, and right child 10?

• 8. If a BST has height h, express search time in terms of h.

• 9. What is the height of a single-node tree under the lecture convention?

• 10. Why must recursive tree traversal include a null check?

18. Answer key

• 1. All map to bucket 7.

• 2. Open addressing and separate chaining.

• 3. 750 / 1000 = 0.75.

• 4. There is no empty slot available for a new distinct entry.

• 5. Search the linked list/collection stored in that bucket.

• 6. Smaller goes left; larger goes right (duplicate policy is implementation-specific).

• 7. Preorder: 8 3 10; inorder: 3 8 10; postorder: 3 10 8.

• 8. O(h).

• 9. Height 0.

• 10. It provides the base case and stops recursion at absent children.

19. Final revision sheet

Concept	Remember this
Hash table	Key–value storage; hash function maps key to slot
put / get	Insert or update / retrieve by key
Collision	Different keys map to same slot
Open addressing	Probe another slot; data stays in table

Concept	Remember this
Separate chaining	Bucket stores a chain/list of entries
Load factor	entries ÷ slots
Tree	Hierarchical node structure with a root
BST	smaller left, larger right
Preorder	Node–Left–Right
Inorder	Left–Node–Right
Postorder	Left–Right–Node
Height convention in this lecture	Leaf height 0; empty tree height −1

Scope note: The uploaded transcript reaches hashing implementation/testing and then tree terminology, BST insertion, and recursive traversals. It mentions further tree topics (such as height calculation, deletion, and completing trees) as future material; those are not treated here as fully covered lecture topics.